

claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_133acdee-dab\agent-a827a6fc81dcdeece.jsonl`  
- SHA-256: `8370b28561de0f99d092827a592640bd83d5c1594b24463ea90cc8ee35598a74`  
- Source modified: `2026-07-06T17:31:53+00:00`  
- Imported at: `2026-07-08T15:59:35+00:00`  
- project: `wf_133acdee-dab`  
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`
```

Transcript

1. user (2026-07-06T17:30:35.317Z)

You are an ADVERSARIAL VERIFIER. Try HARD to REFUTE the following review finding by reading the ACTUAL code in C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (and the diff at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff). Default to REFUTED if you cannot concretely reproduce the defect by pointing at real code lines. A finding is CONFIRMED only if you can trace a concrete input/state that produces the claimed wrong behavior in the code AS WRITTEN (not as you imagine it). If the claimed line/behavior does not exist, or a guard/caller already handles it, or it is out-of-scope speculation, mark REFUTED. If genuinely unclear after reading, mark UNCERTAIN.

FINDING (from the conventions-tests lens):

severity=low | backend/pipeline/segmenters/trajectory_hybrid.py:181

summary: The new process_video docstring overclaims: it says a detector Failure means a failure is 'NOT confused with a genuine 0-rally video', but several sibling non-detection failure paths in the same method still return a bare [] and are written as status="success" by the worker.
detail: The fix converts only the two detector paths (healthcheck-not-ready at line 278, run_predict-no-trajectory at line 296) to Failure. The other early-exit failure paths in the same process_video still return a bare []: bad sport (line 194), missing \${AI_PROVIDER}_API_KEY with skip_ai_handoff=False (line 219), unknown provider (line 226), invalid video duration (line 231), and NotImplementedError for a family without a native runner (line 244). All of these are genuine 'we could not run detection' failures, not 0-rally matches, yet cmd_infer writes them as segment_count=0 / status="success" (worker/__main__.py line 329-330, 369). The missing-key case is the exact silent-success class wf294 targets and is even named in the worker's own 0-segment warning (worker/__main__.py line 343-347). It is materially mitigated ONLY for the cloud-serving profile, and ONLY for the key case, by the AI_HANDOFF_KEY_MISSING boot-time fatal (config/startup.py line 269) ? the other bare-[] paths and other profiles remain uncovered. This is pre-existing behavior (the diff did not introduce it, and test_pipeline_no_api_key_returns_empty locks the []), so it is out of the stated 2-path scope; the reportable defect is that the new comment implies broader silent-failure coverage than the change delivers.

failure_scenario: A worker runs on a profile other than cloud-serving (or a cloud-serving variant that passes startup) with skip_ai_handoff=False and no API key, or with an unknown ai_provider, or a native-only family mismatch: process_video returns []; cmd_infer writes report.json status="success", segment_count=0 and (with --done-uri) a rc-0 done-marker ? the same silent-failure-as-success shape wf294 was meant to eliminate, while the docstring asserts it is handled.

suggested_fix: Either narrow the comment to state only the two detector paths are converted (and the other early-[] returns remain a documented incremental gap), or convert the missing-key / unknown-provider / invalid-duration / native-unavailable paths to Failure as well (updating test_pipeline_no_api_key_returns_empty accordingly).

Verify against the real code and return your verdict.

2. assistant (2026-07-06T17:30:39.872Z)

I'll verify this finding by examining the actual code.

3. user (2026-07-06T17:30:41.940Z)

```
1      diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2      index 503f6d8..f96b495 100644
3      --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4      +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5      @@ -22,7 +22,7 @@ import concurrent.futures
6      import logging
7      import os
8      import time
9      -from typing import Any, Dict, List, Optional, Tuple
10     +from typing import Any, Dict, List, Optional, Tuple, Union
11
12     import cv2
13
14     @@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
15     from backend.pipeline.detectors.base import DetectorRunner
16     from backend.pipeline.segmenters.base import BasePlaySegmenter
17     from backend.providers import provider_registry
18     +
19     +# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
20     +from backend.results import Failure
21     from backend.sports import sport_registry
22     from backend.utils.paths import output_base
23     from backend.utils.video import extract_video_chunk, get_video_duration
24     @@ -174,10 +177,13 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
25         video_path: str,
26         player_pool: Optional[List[str]] = None,
27         max_frames: Optional[int] = None,
28     - ) -> List[Dict[str, Any]]:
29     -     # override-ignore: the ABC declares the Result contract (Success|Failure)
30     -     # but every live segmenter still returns a bare list ? the documented
31     -     # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
32     + ) -> "Union[List[Dict[str, Any]], Failure]":
33     +     # Partial adoption of the Result contract (the ABC declares Success|Failure).
34     +     # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
35     +     # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused
36     +     # genuine 0-rally video (which still returns a bare ``[]``). Every caller
37     +     # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
38     +     # Failure)``;
39     +     # the bare-list success/empty paths are the documented incremental gap
40     +     # (CODE_MAP gotchas).
41     +     label = self.display_label
42     +     # --- Sport profile + AI provider wiring (identical across segmenters) ---
43     +     sport_name = self.config.sport
44     @@ -267,7 +273,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
45     ok, msg = runner.healthcheck()
46     if not ok:
47         logger.error("%s detector environment not ready: %s", label, msg)
48         return []
49         # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
50         # failed/retryable instead of writing an empty-but-"successful" result.
51         return Failure(
52             message=f"{label} detector environment not ready: {msg}",
53             is_recoverable=True,
54         )
55     logger.info("%s detector env OK: %s", label, msg)
```

```

54
55         # --- Phase 2: trajectory -> candidate rally windows ---
56     @@ -278,7 +289,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
57         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
58         if not csv_path:
59             logger.error("%s produced no trajectory; aborting.", label)
60         -         return []
61         +         # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
62         +         # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
63         +         # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
64         +         # rather than a silent 0-segment "success" (the wf294 incident).
65         +         return Failure(
66         +             message=(
67         +                 f"{label} detector produced no trajectory (timed out or aborted) ?
"
68         +                 "detector failure, not a 0-rally video."
69         +             ),
70         +             is_recoverable=True,
71         +         )
72
73         points = runner.parse_trajectory_csv(csv_path)
74         logger.info(
75         diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
76         index 461d7ca..88c50f6 100644
77         --- a/backend/worker/__main__.py
78         +++ b/backend/worker/__main__.py
79         @@ -114,6 +114,42 @@ def _write_report_json(
80             )
81
82
83         +def _serialize_and_push_outputs(
84         +     scratch: str,
85         +     out_uri: str,
86         +     video_id: str,
87         +     segments: List[dict],
88         +     status: str,
89         +     storage_cfg: dict,
90         +     video_uri: str = "",
91         + ) -> List[str]:
92         +     """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
93         +     ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
94         +
95         +     Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
96         +     ``status="failed"`` report.json in the bucket ? the control plane's source of truth
97         +     (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
98         +     report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
99         +     EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. """
100        +     out_local = os.path.join(scratch, "outputs", video_id)
101        +     os.makedirs(out_local, exist_ok=True)
102        +     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
103        +     _write_report_json(
104        +         video_id,
105        +         segments,
106        +         status,
107        +         os.path.join(out_local, "report.json"),
108        +         video_uri=video_uri,
109        +     )
110        +     out_prefix = out_uri.rstrip("/")

```

```

111 +     pushed: List[str] = []
112 +     for fn in sorted(os.listdir(out_local)):
113 +         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
114 +         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
115 +         pushed.append(uri)
116 +     return pushed
117 +
118 +
119     def _run_startup_checks(config: Any) -> bool:
120         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
121         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
122         @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
123             )
124
125         def _fail(rc: int) -> int:
126             -         # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
127             -         # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
128             -         # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
129             +         # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
130             +         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
131             +         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
132             +         # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
133             +         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
134             +         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
135             +         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
136             +         try:
137             +             _serialize_and_push_outputs(
138             +                 scratch,
139             +                 args.out_uri,
140             +                 video_id,
141             +                 [],
142             +                 "failed",
143             +                 storage_cfg,
144             +                 str(getattr(args, "video_uri", "") or ""),
145             +             )
146             +         except Exception as e: # never mask the real failure on an artifact-push
hiccup
147             +             logger.warning("[worker] could not push failure artifacts: %s", e)
148             +             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
149             +             if getattr(args, "done_uri", None):
150             +                 _write_done_marker(
151             +                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
152             @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
153                 )
154
155             # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
156             -         out_local = os.path.join(scratch, "outputs", video_id)
157             -         os.makedirs(out_local, exist_ok=True)
158             -         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
159             -         _write_report_json(
160             +         pushed = _serialize_and_push_outputs(
161             +             scratch,
162             +             args.out_uri,

```

```

163         video_id,
164         segments,
165         status,
166     -     os.path.join(out_local, "report.json"),
167     -     video_uri=str(getattr(args, "video_uri", "") or ""),
168     +     storage_cfg,
169     +     str(getattr(args, "video_uri", "") or ""),
170     )
171
172     -     out_prefix = args.out_uri.rstrip("/")
173     -     pushed = []
174     -     for fn in sorted(os.listdir(out_local)):
175     -         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
176     -         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
177     -         pushed.append(uri)
178     -
179     print(
180         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
181     )
182     diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
183     index 64909e7..c5ae442 100644
184     --- a/tests/test_tracknet_hybrid.py
185     +++ b/tests/test_tracknet_hybrid.py
186     @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
187     from unittest.mock import MagicMock, patch
188
189     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
190     +from backend.results import Failure
191
192
193     def _window(start=1.0, end=4.0):
194     @@ -124,7 +125,10 @@ def test_pipeline_healthcheck_failure_aborts(
195
196         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
197         res = seg.process_video("v1", "clip.mp4")
198     -     assert res == []
199     +     # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
200     +     # instead of writing an empty "success".
201     +     assert isinstance(res, Failure)
202     +     assert "environment not ready" in res.message
203     +     # Should bail before ever running inference.
204     +     mock_runner_cls.return_value.run_predict.assert_not_called()
205
206     @@ -144,7 +148,36 @@ def test_pipeline_no_trajectory_aborts(
207         mock_provider_registry.get.return_value = MagicMock()
208
209         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
210     -     assert seg.process_video("v1", "clip.mp4") == []
211     +     res = seg.process_video("v1", "clip.mp4")
212     +     # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
213     +     # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
214     +     assert isinstance(res, Failure)
215     +     assert "no trajectory" in res.message and res.is_recoverable
216     +
217     +
218     +@_patched
219     +def test_pipeline_zero_windows_is_legit_empty_not_failure(
220     +     mock_runner_cls,
221     +     mock_probe,
222     +     mock_dur,

```

```

223 + mock_extract,
224 + mock_provider_registry,
225 + mock_env,
226 +):
227 + """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
228 + parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
229 + (0-rally "success"), NOT a Failure."""
230 + mock_env.return_value = "dummy_key"
231 + mock_dur.return_value = 30.0
232 + mock_extract.return_value = True
233 + mock_runner_cls.return_value = _make_runner(windows=[]) # trajectory ok, zero
windows
234 + mock_provider_registry.get.return_value = MagicMock()
235 +
236 + seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
237 + res = seg.process_video("v1", "clip.mp4")
238 + assert res == [] and not isinstance(res, Failure)
239 + # The detector DID run (a genuine empty, not an abort).
240 + mock_runner_cls.return_value.run_predict.assert_called_once()
241
242
243 @_patched
244 diff --git a/tests/test_worker.py b/tests/test_worker.py
245 index 96edbee..63cc18e 100644
246 --- a/tests/test_worker.py
247 +++ b/tests/test_worker.py
248 @@ -10,6 +10,7 @@ import json
249 import pytest
250
251 from backend.config import load_config
252 +from backend.results import Failure
253 from backend.worker import __main__ as wmain
254
255 # ----- pure helpers
256 @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
257
258
259 class _EmptySeg:
260 - """A segmenter that finds nothing ? the cold-start failure mode."""
261 + """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""
262
263     def __init__(self, db, config):
264         pass
265 @@ -200,6 +201,20 @@ class _EmptySeg:
266         return []
267
268
269 +class _FailSeg:
270 + """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
271 + trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
272 +
273     def __init__(self, db, config):
274         pass
275
276     def process_video(self, video_id, path, max_frames=None):
277         return Failure(
278             message="WASB detector produced no trajectory (timed out or aborted)",
279             is_recoverable=True,
280         )
281
282
283 def test_setup_logging_levels():

```

```

284         import logging
285
286     @@ -246,12 +261,56 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
287         assert "0 segments" in msgs and "vidZ" in msgs
288         assert "detector_impl=native" in msgs # surfaces the resolved detector
289         assert "native runner UNAVAILABLE" in msgs # points at the likely cause
290     - assert (
291     -         json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
292     -             "segment_count"
293     +         # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
294     +         # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
295     +         report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
296     +         assert report["segment_count"] == 0 and report["status"] == "success"
297     +
298     +
299     +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
300     +     tmp_path, monkeypatch
301     +):
302     +     """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
303     +     as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
304     +     status="failed" so the control plane / alpha-user surface can show an error /
retry."""
305     +     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
306     +     monkeypatch.setattr(
307     +         wmain.validator_registry,
308     +         "run_all_with_context",
309     +         lambda path, cfg: ([_OKResult()], {}),
310     +     )
311     +     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
312     +     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
313     +     video = tmp_path / "in.mp4"
314     +     video.write_bytes(b"FAKE")
315     +     out = tmp_path / "bucket"
316     +     done = tmp_path / "bucket" / "_dispatch" / "tok.done"
317     +
318     +     rc = wmain.main(
319     +         [
320     +             "infer",
321     +             "--video-uri",
322     +             str(video),
323     +             "--out-uri",
324     +             str(out),
325     +             "--scratch",
326     +             str(tmp_path / "s"),
327     +             "--segmenter",
328     +             "wasb_hybrid",
329     +             "--done-uri",
330     +             str(done),
331     +         ]
332     +         == 0
333     +     )
334     +     # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
335     +     assert rc == 3
336     +     # report.json is written as an explicit FAILURE, not a false empty "success".
337     +     report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
338     +     assert report["status"] == "failed" and report["segment_count"] == 0
339     +     assert report["segments"] == []
340     +     # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
341     +     marker = json.loads(done.read_text())

```

```

342 +     assert marker["returncode"] == 3 and marker["status"] == "failed"
343 +     assert marker["segment_count"] == 0 and marker["video_id"] == "vidx"
344
345
346     def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
347

```

4. user (2026-07-06T17:30:43.153Z)

```

160
161     def _select_for_handoff(
162         self,
163         windows: List[Dict[str, Any]],
164         window_stats: List[Dict[str, Any]],
165         idx_cfg: "IndexingConfig",
166     ) -> List[int]:
167         """Indices of the candidate windows that proceed to the AI handoff (and thus may
168         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
169         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
170         Persisted candidates are NOT affected ? they remain the full superset for
offline
171         re-scoring."""
172         return list(range(len(windows)))
173
174     def process_video( # type: ignore[override]
175         self,
176         video_id: str,
177         video_path: str,
178         player_pool: Optional[List[str]] = None,
179         max_frames: Optional[int] = None,
180     ) -> "Union[List[Dict[str, Any]], Failure]":
181         # Partial adoption of the Result contract (the ABC declares Success|Failure).
182         # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
/
183         # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused with
a
184         # genuine 0-rally video (which still returns a bare ``[]``). Every caller
185         # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
Failure)``;
186         # the bare-list success/empty paths are the documented incremental gap (CODE_MAP
gotchas).
187         label = self.display_label
188         # --- Sport profile + AI provider wiring (identical across segmenters) ---
189         sport_name = self.config.sport
190         try:
191             sport_profile = sport_registry.get(sport_name)
192         except KeyError as e:
193             logger.error(str(e))
194             return []
195
196         idx_cfg = self.config.indexing
197         skip_ai = bool(idx_cfg.skip_ai_handoff)
198         provider_name = self.config.ai_provider
199         if skip_ai:
200             model_name = "local-noai"
201             logger.info(
202                 "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
203                 "be emitted as rallies; no %s handoff, no API key needed.",
204                 label,
205                 provider_name,
206             )
207         else:
208             model_name = self.config.ai_model
209             api_key_env_var = f"{provider_name.upper()}_API_KEY"
210             api_key = os.environ.get(api_key_env_var)

```

```

211         if not api_key:
212             from backend.pipeline.segmenters._keyhint import api_key_hint
213
214             logger.error(
215                 f"{api_key_env_var} environment variable is not set! "
216                 f"To run the {provider_name} handoff, set your API key. "
217                 + api_key_hint(api_key_env_var)
218             )
219             return []
220         try:
221             provider = provider_registry.get(
222                 provider_name, api_key=api_key, model_name=model_name
223             )
224         except KeyError as e:
225             logger.error(str(e))
226             return []
227
228         video_duration = get_video_duration(video_path)
229         if video_duration <= 0:
230             logger.error("Invalid video duration.")
231             return []
232         fps, frame_width = self._probe_fps_width(video_path)
233
234         # --- Runner config + windowing thresholds ---
235         # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
236         # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
237         # it delegates to the subclass _build_runner (the real WSL/native runner).
238         try:
239             runner, runner_params = self._resolve_runner(idx_cfg)
240         except NotImplementedError as e:
241             # e.g. detector_impl="native" for a family without a native runner
242             # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
243             # crashing.
244             logger.error("%s: %s", label, e)
245             return []
246
247         velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
248         merge_gap = idx_cfg.dead_time_merge_gap
249         min_window_duration = idx_cfg.min_action_window_duration
250         # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
251         IndexingConfig.
252         max_window_duration = idx_cfg.max_window_duration
253         window_min_split_gap = idx_cfg.window_min_split_gap
254         window_hard_cap = idx_cfg.window_hard_cap
255         window_inactivity_end = idx_cfg.window_inactivity_end
256         inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
257         inactivity_min_density = idx_cfg.inactivity_min_density
258         inactivity_temporal_density = idx_cfg.inactivity_temporal_density
259         window_blob_fallback = idx_cfg.window_blob_fallback
260         window_blob_factor = idx_cfg.window_blob_factor
261         handoff_padding = idx_cfg.ai_handoff_padding
262         ai_max_workers = idx_cfg.ai_max_workers
263         log_candidates = idx_cfg.log_yolo_candidates
264         store_candidates = idx_cfg.store_yolo_candidates
265
266         detection_params = {
267             "detector": self.name,
268             "velocity_thresh": velocity_thresh,
269             "merge_gap": merge_gap,
270             "min_action_window_duration": min_window_duration,
271             **runner_params,
272         }
273
274         # --- Phase 1: env healthcheck (fail fast with a clear message) ---

```

```

273         ok, msg = runner.healthcheck()
274         if not ok:
275             logger.error("%s detector environment not ready: %s", label, msg)
276             # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
277             # failed/retryable instead of writing an empty-but-"successful" result.
278             return Failure(
279                 message=f"{label} detector environment not ready: {msg}",
280                 is_recoverable=True,
281             )
282         logger.info("%s detector env OK: %s", label, msg)
283
284         # --- Phase 2: trajectory -> candidate rally windows ---
285         # Trajectory detectors process the whole video in one pass (they carry
286         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
287         t_start = time.time()
288         out_dir = os.path.join(output_base(self.config), self.family)
289         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
290         if not csv_path:
291             logger.error("%s produced no trajectory; aborting.", label)
292             # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
293             # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
294             # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
295             # rather than a silent 0-segment "success" (the wf294 incident).
296             return Failure(
297                 message=(
298                     f"{label} detector produced no trajectory (timed out or aborted) ? "
299                     "detector failure, not a 0-rally video."
300                 ),
301                 is_recoverable=True,
302             )
303
304         points = runner.parse_trajectory_csv(csv_path)
305         logger.info(
306             "Parsed %d trajectory points (%d visible).",
307             len(points),
308             sum(1 for p in points if p.visible),
309         )
310
311         all_candidate_windows = runner.trajectory_to_action_windows(
312             points,
313             fps=fps,
314             frame_width=frame_width,
315             chunk_start=0.0,
316             velocity_thresh=velocity_thresh,
317             merge_gap=merge_gap,
318             min_window_duration=min_window_duration,
319             chunk_index=0,

```

5. assistant (2026-07-06T17:30:54.344Z)

The finding acknowledges the two detector paths (lines 278, 296) were converted to Failure, and that the other bare-`[]` paths are pre-existing. The core claim is narrower: the new docstring at line 181-186 *overclaims* by implying broader silent-failure coverage than delivered.

Let me examine the actual docstring wording carefully. The comment says:

```

> A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out / aborting with
no trajectory ? returns a `Failure` so it is NOT confused with a genuine 0-rally video (which
still returns a bare `[]`). Every caller ... branches on `isinstance(result, Failure)`; **the
bare-list success/empty paths are the documented incremental gap.**

```

Let me verify the claim by checking whether the docstring actually scopes its claim. The key question: does the comment claim ALL detector failures are Failure, or only the two named ones?

The comment explicitly enumerates: "the env healthcheck not ready, or run_predict timing out / aborting with no trajectory". That is precisely the two paths converted. It does NOT claim the bad-sport, missing-key, unknown-provider, invalid-duration, or NotImplementedError paths return Failure.

Let me check the "0-rally video (which still returns a bare `[]`)" phrase and the "bare-list success/empty paths" phrase against the actual other paths.

6. user (2026-07-06T17:30:55.473Z)

```
1      """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3      This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5      The goal is to make it impossible to confuse "no data / no rallies" with "something
6      actually failed"
7      (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
8
9      We use simple dataclasses so callers can use isinstance checks or structural pattern
10     matching.
11
12     This is intentionally small and incremental ? we start by applying it to the segmenter
13     and provider layers (the biggest sources of silent [] returns today).
14
15     The types are generic where it makes sense. Callers can still access .value on Success
16     or .message on Failure.
17
18     This module has no dependencies on the rest of the app.
19     """
20
21     from __future__ import annotations
22
23     from dataclasses import dataclass, field
24     from typing import Any, Generic, List, Optional, Tuple, TypeVar, Union
25
26     T = TypeVar("T")
27
28     @dataclass
29     class Success(Generic[T]):
30         """Represents a successful outcome with an optional list of non-fatal warnings."""
31
32         value: T
33         warnings: List[str] = field(default_factory=list)
34
35         def is_success(self) -> bool:
36             return True
37
38         def unwrap(self) -> T:
39             """Return the value. Only safe to call if you already know it's a Success."""
40             return self.value
41
42     @dataclass
43     class Failure:
44         """Represents a failure with a human-readable message and optional details."""
45
46         message: str
47         details: Optional[dict[str, Any]] = None
48         is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
49
50         def is_success(self) -> bool:
51             return False
```

```

51
52     def unwrap(self) -> Any:
53         """Raise to make misuse obvious in code that assumes success."""
54         raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57     # Convenient alias for use in type hints: Result[list[dict], Failure]
58     # Result[ValueType] means Success[ValueType] | Failure
59     # (the Failure "error" type is fixed for this contract)
60     Result = Union[Success[T], Failure]
61
62
63     def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64         """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66         - ``Failure``          -> ``(None, failure)``
67         - ``Success(value=v)`` -> ``(v, None)``
68         - a bare value (legacy code that predates this contract and still returns a raw
list)
69         -> ``(value, None)``
70
71         This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72         that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73         on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74         (Success | Failure | legacy bare value)."""
75         if isinstance(result, Failure):
76             return None, result
77         value = result.value if hasattr(result, "value") else result
78         return value, None
79

```

7. assistant (2026-07-06T17:31:11.088Z)

Now let me carefully analyze the docstring's precise claims. The finding hinges on whether the docstring "overclaims."

Reading the docstring at lines 181-186 exactly:

1. Line 182-183: "A DETECTOR failure ? **the env healthcheck not ready, or run_predict timing out / aborting with no trajectory** ? returns a `Failure`" ? This is scoped to EXACTLY the two paths that were converted. It does not say "any detector failure" broadly; it enumerates the two.
2. Line 184: "so it is NOT confused with a genuine 0-rally video (which still returns a bare `[ ]`)" ? the parenthetical clarifies the OTHER return is a genuine 0-rally.
3. Line 186: "the bare-list success/empty paths are the documented incremental gap" ? this explicitly says the bare-list paths remain a gap.

Now, the finding's specific summary claim is: the docstring says a detector Failure means it is "NOT confused with a genuine 0-rally video", but sibling non-detection failure paths (bad sport, missing key, unknown provider, invalid duration, NotImplementedError) still return bare `[ ]` and get written as status="success".

The question is whether those sibling paths are "detector failures" that the docstring claims to cover. Let me examine what the docstring's word "DETECTOR failure" scopes.

The docstring says "A DETECTOR failure ? [enumerated as env healthcheck / run_predict no trajectory]". The bad-sport, missing-key, unknown-provider, invalid-duration paths are NOT detector failures ? they are config/provider/validation failures that occur BEFORE the detector runs. The NotImplementedError (native runner unavailable) is arguably closer to a detector-setup

failure.

Let me check whether the docstring's parenthetical "which still returns a bare `[]`" creates a factual claim that ALL bare-`[]` returns are genuine 0-rally videos. That would be the overclaim.

The phrasing "genuine 0-rally video (which still returns a bare `[]`)" describes the 0-rally case returning `[]`. It does NOT assert the converse ? i.e., it does not claim "every bare `[]` is a genuine 0-rally". And line 186 explicitly labels "the bare-list success/empty paths" (plural, encompassing the non-detector early returns) as "the documented incremental gap."

Let me verify the missing-key path (line 219) and the startup mitigation claim to assess severity, and check the worker to confirm bare `[]` becomes status="success".

8. user (2026-07-06T17:31:12.424Z)

```
300         return rc
301
302     segments: List[dict] = []
303     segmenter_actual = None
304     segmentation_ran = False
305     status = "failed"
306     try:
307         if failures:
308             print(
309                 "[worker] validation FAILED: "
310                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
311             )
312             return _fail(2)
313     seg_name = args.segmenter or config.indexing.default_segmenter
314     segmenter_cls = segmenter_registry.get(seg_name)
315     if not segmenter_cls:
316         print(
317             f"[worker] unknown segmenter: {seg_name!r} "
318             f"(have {list(segmenter_registry.list_available())})"
319         )
320         return _fail(2)
321     segmenter_actual = seg_name
322     result = segmenter_cls(db, config).process_video(
323         video_id, local_video, max_frames=args.max_frames
324     )
325     segmentation_ran = True
326     if isinstance(result, Failure):
327         print(f"[worker] segmenter FAILED: {result.message}")
328         return _fail(3)
329     segments = result.value if hasattr(result, "value") else result
330     status = "success"
331     # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
332     # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
333     # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
334     if not segments:
335         detector_impl = (
336             os.environ.get("RALLY_DETECTOR_IMPL")
337             or getattr(config.indexing, "detector_impl", None)
338             or "auto"
339         )
340         logger.warning(
341             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
342             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
343             "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
```

```

344         "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
345         "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
346         "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
347         "detections, or the input resolution differs from the tuned proxy
resolution. "
348         "Re-run with -v for the native command + frame counts.",
349         video_id,
350         segmenter_actual,
351         detector_impl,
352     )
353     finally:
354         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
355         emit_indexer_report(
356             db=db,
357             video_id=video_id,
358             video_path=local_video,
359             config=config,
360             val_results=val_results,
361             val_context=val_context,
362             segmenter_requested=args.segmenter,
363             segmenter_actual=segmenter_actual,
364             was_reingest=False,
365             segmentation_ran=segmentation_ran,
366         )
367
368     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
369     pushed = _serialize_and_push_outputs(
370         scratch,
371         args.out_uri,
372         video_id,
373         segments,
374         status,

```

9. assistant (2026-07-06T17:31:43.656Z)

This confirms the worker mechanics: a bare `[]` from `process\_video` (line 329) sets `status = "success"` (line 330), and the missing-key / unknown-provider / invalid-duration / NotImplementedError paths all return bare `[]`, so they become status="success" segment_count=0. That part of the finding is factually accurate about the code behavior.

The finding also correctly notes the worker's own warning (lines 343-347) explicitly names "a missing \${AI_PROVIDER}_API_KEY with skip_ai_handoff=False (the segmenter bails BEFORE detection...)" as a cause of a 0-segment "success" ? confirming the missing-key path is indeed a silent-success class.

Now the crux: is the docstring an OVERCLAIM? Let me weigh both readings precisely.

The finding's reportable defect is that "the new comment implies broader silent-failure coverage than the change delivers." Let me test this against the exact words.

The docstring:

- Enumerates the two converted paths precisely (env healthcheck, run_predict no-trajectory).
- Says these return Failure "so it is NOT confused with a genuine 0-rally video (which still returns a bare `[]`)."
- Line 186: "the bare-list success/empty paths are the documented incremental gap (CODE_MAP gotchas)."

The critical phrase is line 186: "***the bare-list success/empty paths are the documented incremental gap**." This phrase explicitly acknowledges that there remain bare-list paths (beyond the genuine-empty success) that are a KNOWN, documented gap. The missing-key, unknown-provider, invalid-duration, bad-sport, and NotImplementedError paths ARE "bare-list ... paths"

and they ARE labeled here as "the documented incremental gap."

So does the docstring "imply broader coverage than delivered"? The finding argues the parenthetical "(which still returns a bare `[]`)" implies every bare `[]` is a genuine 0-rally. But line 186 immediately qualifies that the bare-list paths generally remain a gap. A careful reading does not assert that all bare-`[]` returns are genuine 0-rally videos.

Let me also verify the finding's claim that these are "genuine 'we could not run detection' failures" for the specific paths, and re-check: the docstring's word is "DETECTOR failure." The bad-sport (line 194), missing-key (line 219), unknown-provider (line 226), and invalid-duration (line 231) paths all occur BEFORE any detector/runner is even constructed (runner is resolved at line 239). These are config/provider/input failures, not "DETECTOR failures" in the docstring's sense. The NotImplementedError at line 244 is a runner-resolution failure ? arguably a detector-setup failure, but the docstring specifically enumerates only healthcheck and run_predict, not runner resolution.

Let me confirm the missing-key path is a pre-existing bare-`[]` by checking git blame context isn't needed ? the diff already shows the diff did not touch lines 194/219/226/231/244, so they are pre-existing. The finding concedes this ("pre-existing behavior... out of the stated 2-path scope").

Now let me make the verdict determination.

The finding is ****severity=low**** and its actual reportable defect is a ****documentation/comment precision**** issue: the docstring's parenthetical could be read as implying all bare-`[]` are genuine 0-rally, while several non-detector failure paths also return bare `[]` and surface as status="success".

Assessing adversarially:

****Arguments to REFUTE:****

1. The docstring enumerates exactly the two converted paths ? it does not claim "all detector failures" return Failure.
2. Line 186 explicitly states "the bare-list success/empty paths are the documented incremental gap," directly acknowledging that other bare-list returns remain uncovered. This is the opposite of overclaiming ? it flags the gap.
3. The word "DETECTOR failure" reasonably excludes the pre-detector config/provider/input failures (bad sport, missing key, unknown provider, invalid duration), which occur before the runner exists.
4. The finding itself concedes the behavior is pre-existing and out-of-scope, and that the defect is only that the "comment implies broader silent-failure coverage."

****Arguments to CONFIRM:****

1. The parenthetical "genuine 0-rally video (which still returns a bare `[]`)" can be read as asserting bare-`[]` == genuine 0-rally, which is false for the missing-key case, and the missing-key case is literally the wf294 silent-success class (named in the worker warning).
2. There is genuine tension: the missing-key path is precisely the "silent-success" that wf294 targets, yet it still returns bare `[]` and becomes status="success" ? while the docstring frames the change as addressing the "wf294 silent-failure."

Weighing these: The docstring, read in full including line 186, does NOT actually claim broad coverage. It scopes the Failure conversion to two named paths and explicitly calls the remaining bare-list paths "the documented incremental gap." A reader who reads the whole comment is correctly informed. The "overclaim" only materializes if you read the parenthetical in isolation and ignore line 186. That is a weak, subjective documentation-nuance complaint rather than a concrete defect in code-as-written.

Under the adversarial standard ? default REFUTED unless a concrete wrong behavior is reproducible ? the comment does not produce wrong runtime behavior, and the comment's own line 186 defuses the overclaim reading. The strongest version of the finding is a very mild "the parenthetical could be tightened," which does not rise to a genuine defect given the explicit gap acknowledgment on the next line.

10. user (2026-07-06T17:31:53.543Z)

Structured output provided successfully